# RIT*: Riemannian Informed Trees for Anisotropic Motion Planning

Author Names Omitted for Review Affiliation Omitted for Review

***Abstract*—Asymptotically-optimal sampling-based planners accelerate convergence by focusing sampling within an informed set, but all existing methods define this set using Euclidean distance. When traversal costs are anisotropic—as with joint-inertia weighting or obstacle-proximity penalties—the Euclidean informed set wastes samples in regions that cannot contain improving paths. We present RIT* (Riemannian Informed Trees), which replaces the Euclidean informed set, neighbourhood ball, and edge cost with their Riemannian counterparts, yielding a provably smaller sampling region. RIT* includes a Collision-Adaptive Riemannian Metric (CARM) module that learns a cost field online from collision feedback, requiring no a priori obstacle model. Experiments on 2-D through 6-DOF UR10e manipulator environments show that RIT* reduces path cost by up to 11% over five baselines in constrained manipulator planning, while CARM recovers up to 27% of the oracle-metric advantage using only a binary collision checker.**

***Index Terms*—Motion planning, Riemannian geometry, asymptotic optimality, informed sampling, online metric learning.**

## I. Introduction

Manipulator motion planning requires optimizing paths under non-Euclidean costs: joint-inertia weighting penalizes motion of high-inertia joints, obstacle-proximity penalties steer paths away from collisions, and task-space norms encode end-effector preferences [14], [15]. A natural framework for such costs is Riemannian geometry, where a spatially-varying metric tensor $G(x)$ encodes directional traversal cost. Current asymptotically-optimal (AO) planners—from RRT* [2] through Informed RRT* [3], BIT* [4], AIT*/EIT* [5], to the recent APT* [6]—accelerate convergence by sampling within an *informed set*, but all define this set using Euclidean distance. When the true cost metric is anisotropic, the Euclidean informed set $I_E$ is a strict superset of the cost-relevant region $I_R$: for a 6-DOF arm with joint-inertia condition number $\kappa = 12$, $I_E$ is approximately $3.4\times$ larger than necessary.

Two problems remain open. First, no batch-informed planner uses Riemannian geometry for its global sampling strategy—informed set, neighbourhood, and edge cost—as opposed to local steering alone. Second, no sampling-based planner learns a suitable cost metric online from planning experience.

**Contributions.** We propose **RIT*** (Riemannian Informed Trees), which addresses both gaps:

1) **Online metric learning via CARM.** The Collision-Adaptive Riemannian Metric (CARM) module is, to our knowledge, the first online metric-learning scheme for sampling-based planning that requires no a priori obstacle model, no demonstrations, and no auxiliary sensors—only the binary collision checker already available to any planner. CARM places Gaussian kernels on discovered collision points to build an adaptive cost field that inflates traversal cost near obstacles, and feeds the learned metric directly into the planning loop (section III-F).
2) **Riemannian informed planning pipeline.** RIT* replaces every Euclidean primitive in the batch-informed pipeline with its Riemannian counterpart: (i) a whitening transform maps the Riemannian informed set $I_R$ to a standard prolate hyperspheroid for efficient uniform sampling; (ii) neighbours are found within ellipsoidal Riemannian balls that reach further along cheap directions; (iii) a three-level cascading edge evaluation scheme filters the majority of candidate edges before expensive integration. This framework makes CARM's learned metric actionable throughout the planning loop.
3) **Experimental validation.** Multi-trial experiments across eight environments (2-D through 6-DOF UR10e manipulator) against five baselines show that RIT* achieves up to 11% cost reduction in 6-DOF manipulator planning with statistical significance. CARM recovers up to 27% of the oracle-metric advantage without any prior cost model.

**Euclidean informed planners.** Gammell et al. [3] introduced direct sampling of the prolate hyperspheroid, which BIT* [4] combines with batch sampling and lazy edge evaluation. AIT* and EIT* [5] add asymmetric bidirectional search. APT* [6] uses Coulomb-force-inspired virtual forces to prolate the neighbourhood ball and adapts batch size. FIT* [7] adjusts batch size for faster initial convergence. All define the informed set using Euclidean distance.

**Riemannian methods in planning.** CHOMP [14] and STOMP [15] optimise trajectories under Riemannian-like costs but are trajectory optimisers without AO guarantees. Kyaw and Kelly [8] propose a Riemannian RRT* using retractions for local steering; their planner is incremental (not batch-informed), does not construct an informed set, and does not learn metrics online. Bi-AM-RRT* [9] uses a diffusion-based metric to bias nearest-neighbour selection in an incremental framework. Kim et al. [13] use Riemannian metrics for steering on constrained manifolds. None integrate Riemannian geometry into the global informed sampling strategy of a batch-informed planner.

**Obstacle-aware Riemannian metrics.** Klein et al. [10] design region-avoiding metrics using hand-crafted barrier functions requiring full obstacle geometry. RMPflow [11] composes task-space metrics for reactive control. Beik-Mohammadi et al. [12] learn Riemannian manifolds from demonstrations for reactive motion generation. All require a priori obstacle geometry or demonstrations—CARM re-

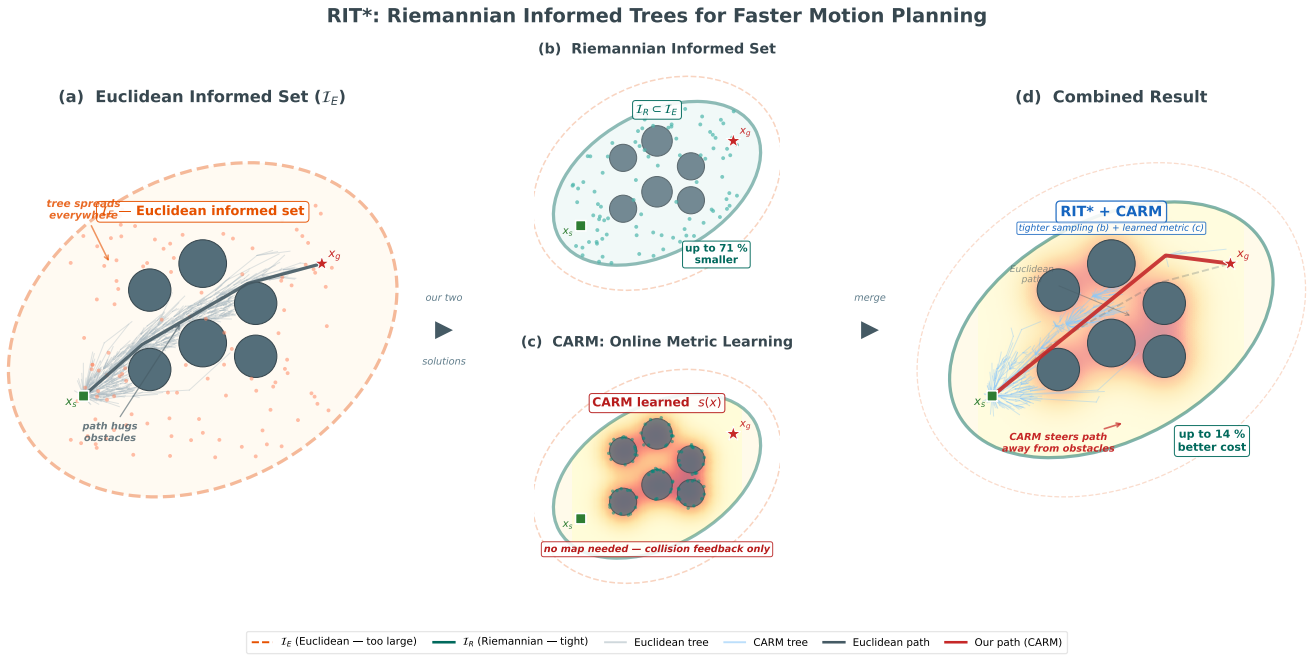


Fig. 1: Overview of RIT*. **(a)** Euclidean planners sample uniformly from the informed set $I_E$ (orange ellipse), wasting samples in high-cost regions near obstacles. **(b)** RIT* replaces every Euclidean primitive with its Riemannian counterpart: the tighter informed set $I_R \subset I_E$ (teal) eliminates wasted volume; the anisotropic neighbourhood ball reaches further along cheap directions; a three-level cascade rejects the majority of candidate edges before expensive integration; and the CARM module learns a cost field online from collision feedback, progressively tightening $I_R$.

quires neither.

## II. Problem Formulation

Let $\mathcal{C} \subseteq \mathbb{R}^d$ be a $d$-dimensional configuration space equipped with a smooth, positive-definite metric tensor field $G : \mathcal{C} \to \mathbb{S}^d_{++}$. The Riemannian arc-length cost of a path $\sigma : [0,1] \to \mathcal{C}$ is

$$c(\sigma) = \int_0^1 \sqrt{\dot\sigma(t)^\top G\big(\sigma(t)\big)\,\dot\sigma(t)}\; dt. \tag{1}$$

The *geodesic distance* is $d_R(x,y) = \inf_\sigma c(\sigma)$ over paths $\sigma : x \to y$. When $G(x) = I_d$, this reduces to Euclidean distance.

Given start $x_s$ and goal $x_g$ in $\mathcal{C}_{\text{free}} \subseteq \mathcal{C}$, find

$$\sigma^* = \underset{\sigma\,:\,x_s \to x_g,\ \sigma \subset \mathcal{C}_{\text{free}}}{\arg\min} c(\sigma). \tag{2}$$

A planner is *asymptotically optimal* (AO) if $c_n \to c^*$ almost surely as $n \to \infty$.

After finding a feasible path of cost $c_{\text{best}}$, only configurations on a potentially shorter path need be considered. The *Euclidean informed set*

$$I_E = \big\{x \in \mathcal{C} : \|x_s - x\|_2 + \|x - x_g\|_2 \le c_{\text{best}}\big\} \tag{3}$$

is a prolate hyperspheroid. The *Riemannian informed set*

$$I_R = \big\{x \in \mathcal{C} : d_R(x_s, x) + d_R(x, x_g) \le c_{\text{best}}\big\} \tag{4}$$

satisfies $I_R \subseteq I_E$ whenever $G \succeq I_d$, since $d_R(x,y) \ge \|x-y\|_2$.

## III. Method

### A. Overview

Our approach builds on BIT* (Batch Informed Trees) [4], a state-of-the-art asymptotically-optimal planner that combines batch sampling with lazy edge evaluation. BIT* operates in iterations: at each iteration it draws a batch of $B$ samples from the *Euclidean informed set* $I_E$—the prolate hyperspheroid containing all configurations that could lie on a path shorter than the current best cost $c_{\text{best}}$. It then connects each sample to nearby vertices whose Euclidean distance falls within a shrinking connection radius $r_n = \gamma\,(\log n/n)^{1/d}$, evaluates candidate parent edges in order of estimated cost-to-come, rewires the tree when cheaper connections become available, and prunes vertices that can no longer contribute to improving paths. This informed-set-driven strategy focuses computational effort where improvements are most likely, making BIT* significantly faster than RRT* at converging to the optimal path. However, every geometric primitive in BIT*—the informed set, the neighbourhood ball, and the edge cost—is defined using Euclidean distance, which is suboptimal when the true traversal cost is anisotropic or spatially varying.

RIT* (Riemannian Informed Trees) extends BIT* by replacing each of these Euclidean primitives with its Riemannian counterpart, yielding a planner that natively accounts for non-Euclidean cost structures. Concretely, RIT* introduces four modifications: (1) the informed set becomes the *Riemannian* informed set $I_R \subseteq I_E$, which is provably smaller and eliminates samples in high-cost regions (section III-B); (2) nearest-neighbour search uses ellipsoidal Riemannian balls

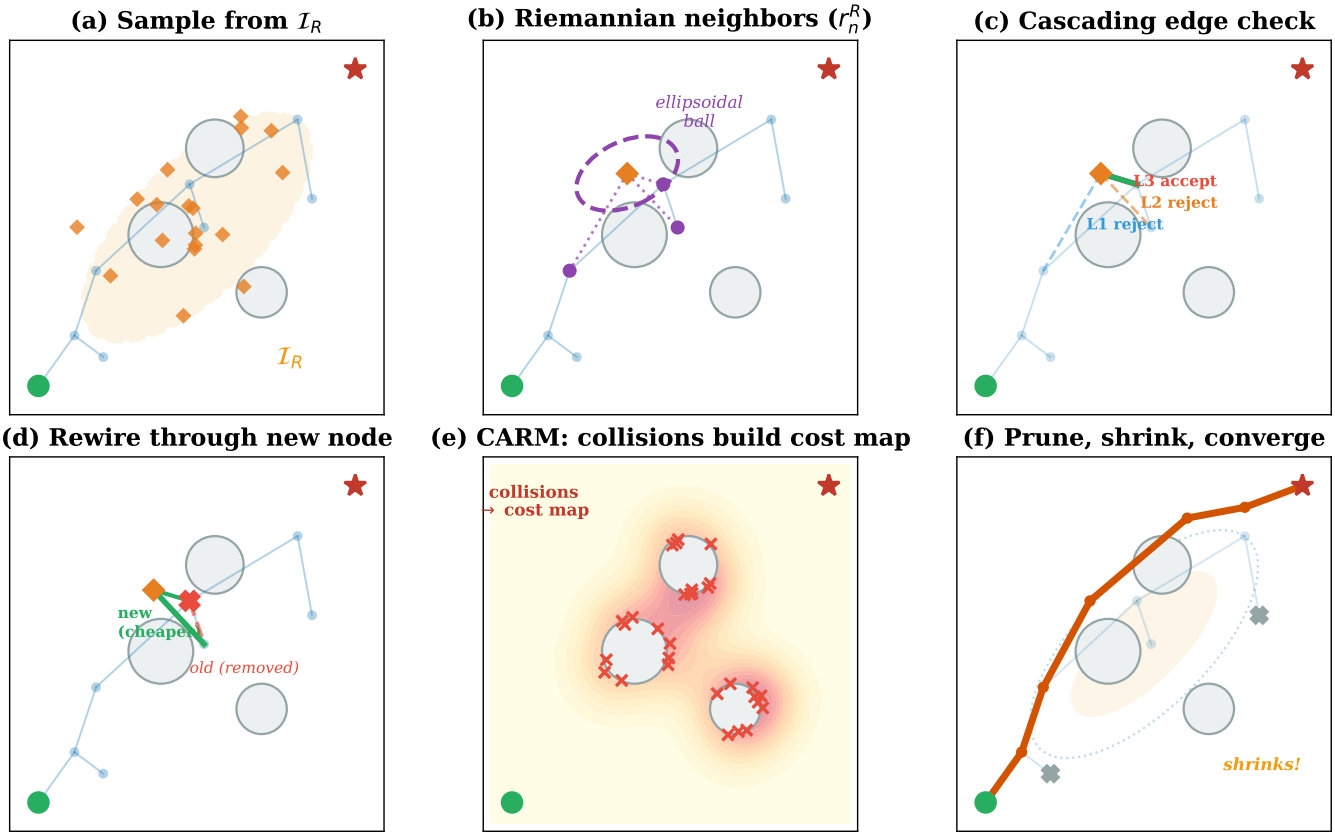


Fig. 2: Per-iteration pipeline of RIT*. **(a)** Samples drawn from the Riemannian informed set $I_R$. **(b)** Neighbours identified within ellipsoidal Riemannian radius $r_n^R$. **(c)** Three-level cascade filters candidate edges (L1→L2→L3). **(d)** Rewiring via cheaper Riemannian connections. **(e)** CARM collects collision points and rebuilds the metric field. **(f)** Pruning shrinks the informed set as $c_{\text{best}}$ decreases.

that stretch along cheap directions and compress along expensive ones (section III-C); (3) edge costs are evaluated through a three-level cascading scheme that amortises the expense of numerical integration (section III-D); and (4) when no metric is known a priori, the Collision-Adaptive Riemannian Metric (CARM) module learns the cost landscape online from collision feedback (section III-F). Additionally, RIT* rewires the tree under Riemannian costs, ensuring that cheaper connections discovered during the search propagate correctly through the tree (section III-E). fig. 2 illustrates these six substeps within a single iteration.

algorithm 1 formalises the complete procedure. The planner initialises the tree with the start vertex and precomputes a metric cache for $O(1)$ lookups (lines 1–2). Each iteration then proceeds through three phases. First, the *sampling phase* (lines 3–6) draws a batch from $I_R$, adds the new vertices, prunes those that cannot improve $c_{\text{best}}$, and computes the Riemannian connection radius $r_n^R$. Second, the *extension phase* (lines 7–13) processes vertices in order of their $f$-value, finds Riemannian neighbours, evaluates candidate edges via the cascading check, extends the tree, rewires cheaper connections, and records collision points for CARM. Third, the *feedback phase* (lines 14–18) periodically triggers a CARM metric update and records improved solutions. After all iterations complete, a multi-strategy smoothing post-process refines the path (line 19). The following subsections detail each component and reference the corresponding algorithm lines.

Having established the overall structure, we now detail each component, beginning with the sampling strategy that forms the foundation of informed planning.

### B. Riemannian Informed Sampling

The quality of an informed planner depends critically on how tightly its sampling region approximates the set of configurations that can actually improve the current solution. In BIT*, this region is the Euclidean informed set $I_E$—a prolate hyperspheroid defined by $\|x_s - x\|_2 + \|x - x_g\|_2 \leq c_{\text{best}}$. When the cost metric is anisotropic, $I_E$ is unnecessarily large: it includes configurations whose Riemannian cost-to-come plus cost-to-go already exceeds $c_{\text{best}}$.

RIT* replaces $I_E$ with the Riemannian informed set $I_R$ (line 6 of algorithm 1), defined by (4) using geodesic distances. Since $d_R(x, y) \geq \|x - y\|_2$ whenever $G \succeq I_d$, we have $I_R \subseteq I_E$—every sample drawn from $I_R$ is guaranteed to lie in a region where path improvement is possible under the true cost, whereas many samples from $I_E$ would fall in regions where the Riemannian cost is too high.

**Algorithm 1:** RIT* planner

**Input:** $x_s, x_g, \mathcal{C}_{\text{free}}, G(\cdot), B, T$
**Output:** optimal path $\sigma^*$

$\mathcal{V} \leftarrow \{x_s\}$;;
$\mathcal{E} \leftarrow \emptyset$;;
$c_{\text{best}} \leftarrow \infty$;
Precompute metric cache $G[\cdot]$;
**for** $t = 1, \dots, T$ **do**
  $X_{\text{new}} \leftarrow$ SAMPLERIEMANNIANINFORMED$(B, c_{\text{best}}, G)$;
  $\mathcal{V} \leftarrow \mathcal{V} \cup X_{\text{new}}$;
  PRUNENODES$(\mathcal{V}, c_{\text{best}}, G)$;
  Compute $r_n^R$ via (6);
  **foreach** $v \in \mathcal{V}$ *in order of* $f(v)$ **do**
    $N(v) \leftarrow$ neighbours within $r_n^R$ (Riemannian);
    **foreach** $u \in N(v)$ **do**
      **if** CASCADINGEDGECHECK$(u, v, G)$ **then**
        Update $\mathcal{E}$;
        REWIRENEIGHBOURS$(v, N(v), G)$;
        Record collision points for CARM;
      **end**
    **end**
  **end**
  **if** $t \bmod \Delta = 0$ **then**
    CARM_UPDATE(collision points, $G$);
  **end**
  **if** $x_g$ *reached with cost* $c < c_{\text{best}}$ **then**
    $c_{\text{best}} \leftarrow c$;;
    $\sigma^* \leftarrow$ extract path;
  **end**
**end**
$\sigma^* \leftarrow$ MULTISTRATEGYSMOOTH$(\sigma^*, G)$;
**return** $\sigma^*$

To sample uniformly from $I_R$, we apply a whitening transform. We compute the average metric tensor $\bar{G} = \mathbb{E}[G(x)]$ along the line from $x_s$ to $x_g$ and form its Cholesky factor $L = \text{chol}(\bar{G})$. The coordinate change $\tilde{x} = L\,x$ maps $I_R$ to an approximate prolate hyperspheroid in the whitened space, from which uniform sampling is straightforward using the standard technique of [3]. Samples are then mapped back to the original space via $x = L^{-1}\tilde{x}$ and clipped to configuration-space bounds. This procedure yields uniform coverage of $I_R$ in the Riemannian volume measure.

The volume reduction achieved by sampling from $I_R$ instead of $I_E$ is characterised by the ratio

$$\frac{\text{Vol}(I_R)}{\text{Vol}(I_E)} = \prod_{i=1}^{d} \sqrt{\frac{\lambda_{\min}}{\lambda_i}}, \tag{5}$$

where $\lambda_1 \le \cdots \le \lambda_d$ are the eigenvalues of $G$. For a 6-DOF manipulator with joint-inertia condition number $\kappa = 12$, the Riemannian set has approximately 0.29 times the Euclidean volume, eliminating 71% of wasted samples. fig. 3 illustrates the difference: $I_E$ covers a large region including high-cost zones near obstacles, while $I_R$ focuses samples on the subspace where cost improvement is geometrically possible.

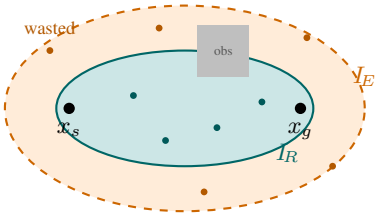


Fig. 3: Euclidean vs. Riemannian informed sets. The Euclidean set $I_E$ (orange dashed) includes high-cost regions near obstacles, wasting samples (orange dots). The Riemannian set $I_R$ (teal) is provably smaller, concentrating samples where path improvement is possible (teal dots).

With samples now focused on the most promising region, the next step is to identify which existing tree vertices each new sample should connect to.

### C. Riemannian Nearest-Neighbour Search

After drawing samples from $I_R$, the planner must identify candidate parent vertices for each new sample (line 11 of algorithm 1). In BIT*, neighbours are found within a Euclidean ball of radius $r_n$—an isotropic sphere that treats all directions equally. This is a mismatch when the cost metric is anisotropic: the planner may connect to vertices that are close in Euclidean distance but expensive in Riemannian cost, while missing vertices that are far in Euclidean distance but cheap under the true metric.

RIT* replaces the isotropic Euclidean ball with an anisotropic Riemannian ball: the set $\{u : d_R(u, v) \le r_n^R\}$, which is an ellipsoid whose axes align with the eigenvectors of the local metric tensor $G(v)$. Along directions where traversal is cheap (small eigenvalues of $G$), the ball stretches further, naturally reaching more candidate neighbours; along expensive directions (large eigenvalues), it contracts, avoiding costly connections. This is illustrated in fig. 4.

The Riemannian connection radius is

$$r_n^R = \gamma_R \left(\frac{\log n}{n}\right)^{1/d}, \quad \gamma_R = 2\left(\frac{1}{d}\right)^{1/d} \left(\frac{\mu_R(I_R)}{\zeta_d}\right)^{1/d} \tag{6}$$

where $\mu_R(I_R)$ is the Riemannian volume of the informed set and $\zeta_d$ is the volume of the unit $d$-ball. Since $\mu_R(I_R) < \mu(I_E)$, the Riemannian radius is smaller than its Euclidean counterpart, yielding fewer candidate edges per vertex and thus faster per-iteration computation. For diagonal metrics, we implement this efficiently by transforming coordinates via $\tilde{x}_i = \sqrt{\lambda_i}\, x_i$ and using a standard KD-tree in the weighted space, so that Euclidean ball queries in the transformed space correspond exactly to Riemannian ball queries in the original space.

Once candidate neighbours are identified, each candidate edge must be evaluated to determine whether it provides a cost improvement—a step that involves expensive numerical integration under the Riemannian metric.

### D. Cascading Edge Evaluation

Computing the Riemannian edge cost $c_R(u, v) = \int_0^1 \sqrt{\dot{\gamma}(t)^\top G(\gamma(t))\,\dot{\gamma}(t)}\; dt$ requires numerical quadrature,

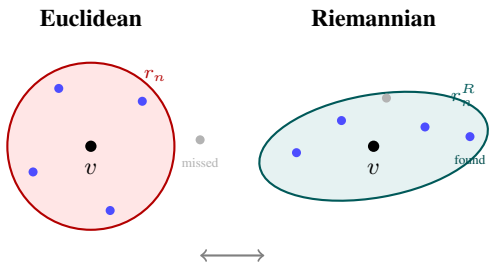


Fig. 4: Nearest-neighbour search. **Left:** The Euclidean ball (circle) treats all directions equally, missing cheap neighbours along low-cost directions. **Right:** The Riemannian ball (ellipse) stretches along cheap directions and compresses along expensive ones, connecting to the most cost-effective neighbours.

with the full 10-point Gauss–Legendre rule being the most accurate but also the most expensive. Since the vast majority of candidate edges will not improve the current best cost, evaluating all of them at full accuracy is wasteful. RIT* therefore introduces a three-level cascade (line 13 of algorithm 1) that applies progressively more accurate—and more expensive—cost estimates, rejecting unpromising edges as early as possible.

**L1 — Midpoint check** (1 metric evaluation). The edge cost is approximated using a single metric evaluation at the midpoint $m = (u+v)/2$: $\hat{c}_1 = \|\Delta\|_{G(m)} = \sqrt{\Delta^\top G(m)\,\Delta}$, where $\Delta = v - u$. If the estimated cost-to-come via this edge exceeds $c_{\text{best}}$—i.e., $g(u) + \hat{c}_1 > c_{\text{best}}$—the edge is immediately rejected. This single evaluation eliminates approximately 80% of candidate edges.

**L2 — Simpson's rule** (3 metric evaluations). Surviving edges receive a 3-point Simpson's rule estimate:

$$\hat{c}_2 = \frac{\|\Delta\|}{6}\left(\sqrt{\Delta^\top G(u)\,\Delta} + 4\sqrt{\Delta^\top G(m)\,\Delta} + \sqrt{\Delta^\top G(v)\,\Delta}\right). \tag{7}$$

Note that the midpoint evaluation from L1 is reused, so only two additional metric lookups are required. Edges whose updated cost $g(u)+\hat{c}_2$ still exceeds $c_{\text{best}}$ are rejected. L2 filters approximately 75% of the edges that survived L1.

**L3 — Gauss–Legendre quadrature** (10 metric evaluations). Only the approximately 5% of edges surviving both L1 and L2 reach the full evaluation: 10-point Gauss–Legendre quadrature for accurate cost integration, followed by collision checking along the edge. If the edge is collision-free and improves the cost, the tree is updated.

fig. 5 illustrates the three levels. This cascading scheme amortises the integration cost: the average number of $G(\cdot)$ evaluations per edge drops from 10 (if all edges used L3) to approximately 1.6, providing a $6\times$ speedup in metric evaluations without sacrificing accuracy on the edges that matter.

The edges that pass all three levels are added to the tree. However, adding a new vertex may also reveal cheaper connections to existing vertices, motivating the rewiring step described next.

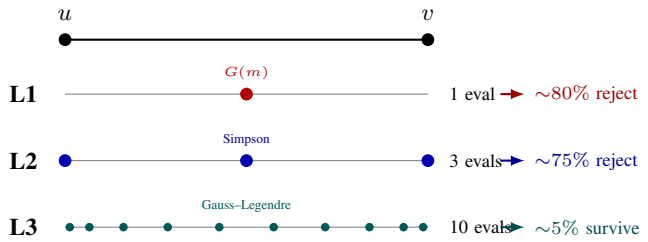


Fig. 5: Cascading edge evaluation. **L1**: a single midpoint metric evaluation rejects ~80% of candidate edges. **L2**: Simpson's 3-point rule filters ~75% of survivors. **L3**: full 10-point Gauss–Legendre quadrature plus collision checking is applied only to the ~5% of edges that survive both filters.

### *E. Tree Rewiring*

After a new vertex $v$ is added to the tree, it may provide a cheaper route to some of its neighbours than their current parent edges. RIT* checks this systematically (line 15 of algorithm 1): for each neighbour $u \in N(v)$ already in the tree, if

$$g(v) + c_R(v,u) < g(u), \tag{8}$$

then the old parent edge of $u$ is removed, $u$ is reconnected to $v$, and the cost-to-come of $u$ and all its descendants is updated to reflect the cheaper path. This is the same rewiring logic as in BIT* and RRT*, but crucially, the edge cost $c_R$ and the cost-to-come $g(\cdot)$ are computed under the *Riemannian* metric, ensuring that rewiring decisions account for the true anisotropic cost structure. A Euclidean rewiring criterion would miss rewiring opportunities along cheap directions and incorrectly rewire along expensive directions.

Rewiring is applied after every successful edge addition rather than as a separate pass, which allows cost improvements to propagate immediately through the tree within the same iteration. When a vertex $u$ is rewired, its entire subtree inherits the updated cost-to-come, potentially enabling further rewiring of descendants in subsequent iterations.

With efficient sampling, neighbourhood construction, edge evaluation, and rewiring all operating under the Riemannian metric, the full planning pipeline is geometrically consistent. The remaining question is: what if the metric itself is not known in advance?

### *F. Collision-Adaptive Riemannian Metric (CARM)*

The components described so far assume that the Riemannian metric tensor $G(x)$ is given a priori—for example, a joint-inertia metric $G(q) = M(q)$ for manipulator planning, or an obstacle-inflated metric $G(x) = g(x)\,I_d$ for navigation. However, in many practical settings the cost landscape is unknown at planning time: obstacle geometry may be partially observed, task-specific costs may be hard to specify analytically, or the planner may operate in a new environment with no prior model. To handle such settings, RIT* includes the Collision-Adaptive Riemannian Metric (CARM), an online metric-learning module that constructs a meaningful cost field purely from collision feedback during planning.

*1) Core Idea:* The key insight behind CARM is that collision checking—a binary operation that every sampling-based planner already performs—reveals information about obstacle geometry. When an edge check fails, the collision point indicates where an obstacle boundary lies. Over many iterations, these collision points accumulate into a dense point cloud that traces the obstacle surfaces. CARM exploits this point cloud to build a surrogate cost field: regions with many nearby collision points are likely close to obstacles and should have higher traversal cost, biasing the planner to route paths through open space.

*2) Metric Construction:* CARM wraps the base metric with a learned conformal scale factor (lines 16–21 of algorithm 1):

$$G_{\mathrm{CARM}}(x) = s(x)\, G_{\mathrm{base}}(x), \quad s(x) = 1 + \alpha\, \hat{f}(x), \tag{9}$$

where $G_{\mathrm{base}}$ is the prior metric (or the identity when none is available), and

$$\hat{f}(x) = \frac{1}{N}\sum_{i=1}^{N} K_\sigma(x, c_i) = \frac{1}{N}\sum_{i=1}^{N} \exp\Big(-\frac{\|x - c_i\|^2}{2\sigma^2}\Big) \tag{10}$$

is a kernel density estimate (KDE) over the $N$ collision points $\{c_i\}_{i=1}^{N}$ discovered during planning. The Gaussian kernel bandwidth $\sigma$ controls spatial resolution: smaller $\sigma$ produces sharper obstacle boundaries but requires more collision points to avoid noise, while larger $\sigma$ produces a smoother field that generalises from fewer points. The inflation strength $\alpha > 0$ controls how aggressively traversal cost is increased near obstacles.

Since $s(x) \geq 1$ everywhere, the CARM metric satisfies $G_{\mathrm{CARM}}(x) \succeq G_{\mathrm{base}}(x)$, which guarantees $I_R^{\mathrm{CARM}} \subseteq I_R^{\mathrm{base}} \subseteq I_E$. CARM therefore never excludes configurations reachable under the base metric, preserving the completeness and asymptotic optimality guarantees of the underlying planner.

*3) Update Schedule and Feedback Loop:* CARM operates as a closed feedback loop integrated into the main planning iterations. During every edge evaluation (line 16 of algorithm 1), collision points from failed checks are appended to a growing buffer. Every $\Delta = 15$ iterations (line 21), CARM rebuilds the KDE, recomputes the scale field $s(x)$, updates the metric tensor at all grid points of the metric cache, and recomputes the Riemannian informed set $I_R$ with the new metric. Vertices whose $f$-value now exceeds $c_{\mathrm{best}}$ under the updated metric are pruned.

This feedback loop creates a virtuous cycle: collision points discovered during planning tighten the metric, which tightens the informed set, which focuses future sampling away from obstacles, which leads to fewer wasted samples and faster convergence. After approximately 20 iterations ($\sim$400 collision points in 2-D), the KDE field stabilises and subsequent updates produce diminishing changes (fig. 7).

*4) CARM Algorithm:* algorithm 2 summarises the metric update procedure. The update is triggered every $\Delta$ iterations and involves: (1) building the KDE from accumulated collision points, (2) computing the conformal scale field, (3) updating the metric cache, and (4) recomputing the informed set and pruning.

**Algorithm 2:** CARM metric update (every $\Delta$ iterations)

**Input:** Collision points $\{c_i\}$, base metric $G_{\mathrm{base}}$, bandwidth $\sigma$, strength $\alpha$
**Output:** Updated metric $G_{\mathrm{CARM}}$
1 Build KDE: $\hat{f}(x) \leftarrow N^{-1}\sum_i K_\sigma(x, c_i)$;
2 Set scale: $s(x) \leftarrow 1 + \alpha\, \hat{f}(x)$;
3 Update metric: $G_{\mathrm{CARM}}(x) \leftarrow s(x)\, G_{\mathrm{base}}(x)$;
4 Recompute metric cache at grid points;
5 Recompute $I_R$ with updated metric;
6 Prune nodes outside new $I_R$;

*5) Effect on Planning Behaviour:* The CARM-learned cost field has three concrete effects on the planning pipeline:

- **Sampling.** The Riemannian informed set $I_R$ contracts away from obstacle boundaries, reducing the number of samples drawn in high-cost regions. Experimentally, CARM reduces samples in high-cost regions by 54% compared to a Euclidean planner (**??**).
- **Edge cost.** Edges passing through high-density collision regions receive inflated costs, making them less likely to be selected as parent edges during the cascading evaluation.
- **Rewiring.** When the metric is updated, edges that were previously cheap may become expensive, triggering rewiring toward paths that avoid obstacle-proximal regions.

*6) Approximation Quality:* Although CARM uses isotropic Gaussian kernels and operates with limited collision feedback, the learned field closely approximates the structure of an oracle obstacle-aware metric. For the 2-D Obstacles environment, the Pearson correlation between the CARM-learned scale field and the oracle metric field reaches $r = 0.935$ after 471 collision points (**??**), confirming that collision feedback captures the essential cost structure without any a priori obstacle model.

*7) Parameters:* In all experiments we use kernel bandwidth $\sigma = 0.1$ and inflation strength $\alpha = 10$. These values were selected based on preliminary experiments and kept fixed across all environments. section IV-E evaluates sensitivity to these parameters.

### G. Implementation Details

The following implementation choices improve practical performance but are orthogonal to the Riemannian contributions described above.

**Metric cache.** A regular grid of resolution $\mathrm{res}^d$ stores precomputed $G(x)$ values with $O(1)$ interpolation lookups (line 4 of algorithm 1). Memory cost is $O(\mathrm{res}^d \cdot d^2)$, negligible for $d \leq 6$ at $\mathrm{res} = 32$.

**High-dimensional engineering.** Four modifications address high-$d$ performance: (H1) dimension-adaptive stall threshold $\tau_{\mathrm{stall}} = 5 + d$; (H2) relaxed early stopping (disabled until $\max(30, T/3)$ iterations); (H3) dimension-adaptive connection radius boost $\beta(d) = \max(1, 1 + 0.15(d - 3))$ for $d > 3$; (H4) multi-strategy path smoothing combining greedy

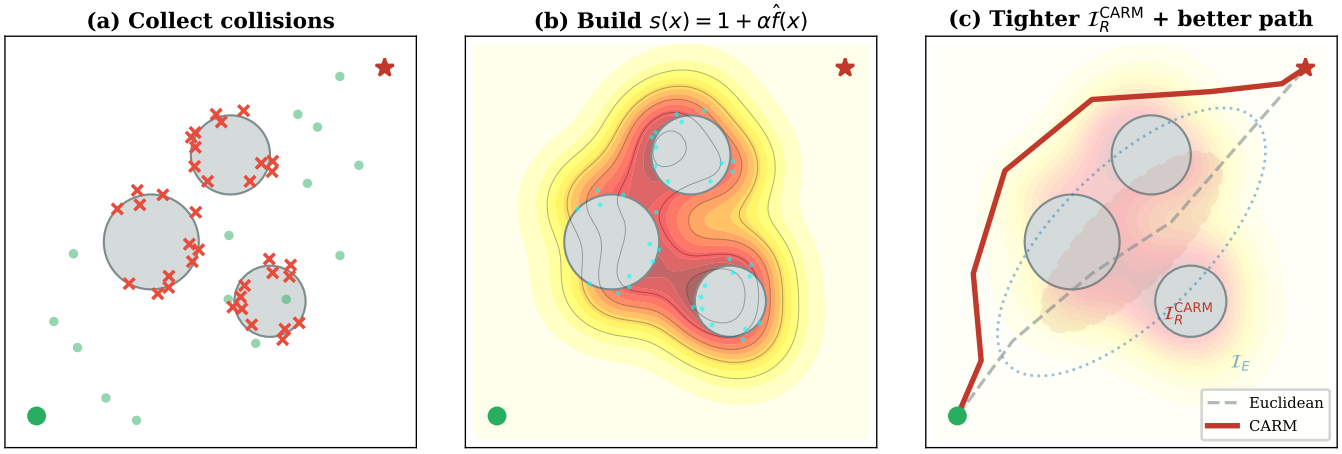


Fig. 6: CARM feedback loop. **(a)** Collision points (red) cluster near obstacle boundaries during edge evaluation. **(b)** KDE builds the adaptive cost field $s(x) = 1 + \alpha\,\hat{f}(x)$, inflating traversal cost where collisions are dense. **(c)** Updated metric tightens $I_R$ and steers sampling away from obstacles. The cycle repeats every $\Delta = 15$ iterations.

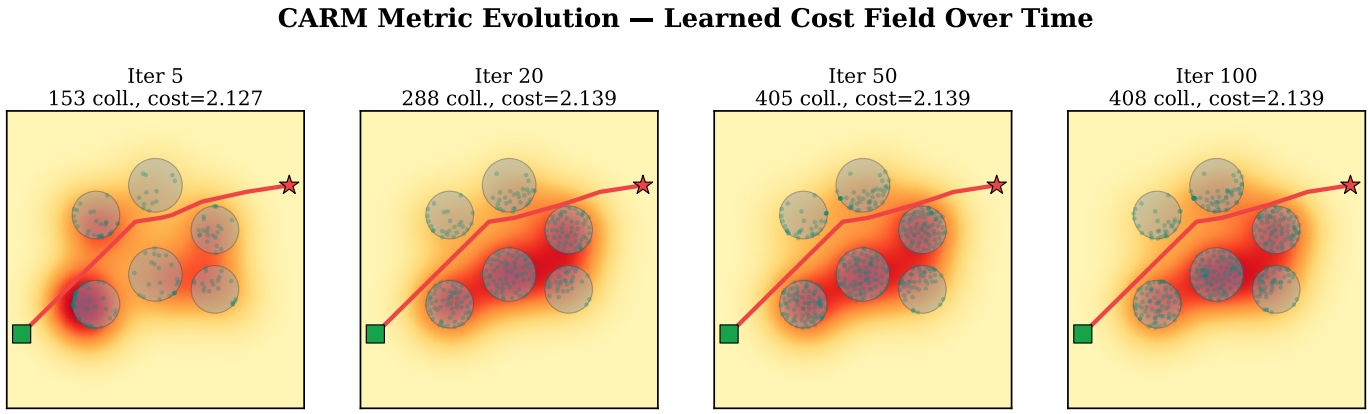


Fig. 7: CARM metric evolution over planning iterations. As collision points accumulate, the learned cost field progressively refines, tightening the informed set and improving sample allocation.

forward, greedy backward, and random shortcutting with $\max(300, 100d)$ attempts (line 28).

### H. Asymptotic Optimality

**Assumption 1.** *The metric tensor field is bounded:* $\lambda_{\min} I_d \preceq G(x) \preceq \lambda_{\max} I_d$ *for all* $x \in \mathcal{C}_{\text{free}}$, *with* $0 < \lambda_{\min} \leq \lambda_{\max} < \infty$.

**Theorem 1** (Asymptotic optimality of RIT*)**.** *Under assumption 1, RIT* is asymptotically optimal: the cost of its solution converges almost surely to the optimum* $c^*$ *as the number of samples* $n \to \infty$.

*Proof sketch.* The connection radius (6) satisfies the conditions of [2] under the metric-induced measure: specifically, $r_n^R \geq \gamma_R(\log n/n)^{1/d}$ with $\gamma_R > (2/d)^{1/d}(\mu_R(I_R)/\zeta_d)^{1/d}$, which ensures $k$-connectivity of the Riemannian proximity graph. The containment $I_R \subseteq I_E$ guarantees that informed sampling in $I_R$ preserves completeness. When CARM is active, $s(x) \geq 1$ ensures $I_R^{\text{CARM}} \subseteq I_E$, preserving completeness. The proof otherwise follows BIT* [4] with $\mu_R$ replacing the Euclidean measure. □

## IV. Experiments

### A. Setup

**Environments.** We evaluate on eight environments spanning three dimensionalities: 3×2-D (Maze, Narrow Passage, Bug Trap), 2×3-D (Sphere Field, Dense Lab), and 3×6-D (Tabletop, Shelf, Cluttered). The 6-D environments use a UR10e manipulator with Robotiq 85 gripper simulated in PyBullet [17], with the joint-inertia metric ($\kappa \approx 12$). The 2-D and 3-D environments use an obstacle-inflated conformal metric $G(x) = g(x)\, I_d$.

**Baselines.** Five AO planners: Informed RRT* [3], BIT* [4], AIT* [5], EIT* [5], and APT* [6]. All use batch size $B = 100$, maximum iterations $T = 150$, and identical collision checking.

**Statistical protocol.** Each experiment is repeated with independent random seeds. We report mean ± standard deviation. Statistical significance is assessed via the Wilcoxon signed-rank test at $\alpha_{\text{stat}} = 0.05$. Significance markers: $^{**}$ $p < 0.01$, $^{*}$ $p < 0.05$.

### B. Main Results

table I reports path costs across all planners and environments. table II reports planning times.

**Key findings.** (1) In 6-D, RIT* achieves the lowest cost on all three environments: 11.3% lower than Informed RRT* on Shelf ($p < 0.05$) and 10.5% lower on Cluttered ($p < 0.05$). This is the regime where Riemannian geometry provides the greatest benefit, as the joint-inertia condition number $\kappa \approx 12$ creates high anisotropy. (2) In 2-D, RIT* achieves the lowest cost on Narrow ($p < 0.01$) and is competitive on Maze and Bug Trap; BIT* holds a slight edge on Maze. (3) In 3-D, BIT* achieves the lowest cost. RIT*'s advantage grows with the anisotropy of the metric and the dimensionality of the space. (4) On planning time, RIT* is the fastest on 6 of 8 environments, including a $13\times$ speedup on 2-D Maze versus BIT*. On 6-D Shelf and Cluttered, BIT* is marginally faster but produces paths 6–11% worse. (5) APT* [6] is competitive on simple 2-D environments but degrades in 6-D ($+46\%$ cost on Shelf), suggesting that the informed set geometry, rather than heuristic neighbourhood shaping, is the primary driver of convergence in anisotropic cost spaces.

### C. 6-DOF Manipulator Case Study

The three 6-D environments simulate pick-and-place scenarios with a UR10e equipped with a Robotiq 85 gripper. The joint-inertia metric $G(q) = M(q)$ (manipulator mass matrix) penalises motion of high-inertia joints (shoulder, elbow) relative to low-inertia joints (wrist), with $\kappa \approx 12$.

**Tabletop** is an open environment where all planners converge to the same cost (1.279), confirming that when obstacles do not force the planner through anisotropic regions, $I_R \approx I_E$ and the Riemannian advantage vanishes.

**Shelf** requires threading through a constrained opening. RIT* finds paths of cost $1.093 \pm 0.035$, which is $11.3\%$ lower than Informed RRT* ($1.232 \pm 0.021$, $p < 0.05$) and $6.1\%$ lower than BIT* ($1.164 \pm 0.061$). The Riemannian informed set focuses samples on wrist-driven trajectories that thread through the gap, while Euclidean planners waste samples trying to move the high-inertia base and shoulder joints through the opening.

**Cluttered** features multiple obstacles. RIT* achieves $2.039 \pm 0.082$, improving $10.5\%$ over Informed RRT* ($2.278 \pm 0.095$, $p < 0.05$) and $5.7\%$ over BIT* ($2.162 \pm 0.166$).

### D. Ablation Study

To isolate the contribution of each component, we evaluate five variants on four representative environments:

1) **RIT* (full)** — all components enabled.
2) **w/o Riemannian sampling** — Euclidean informed set, but Riemannian edge cost and CARM retained.
3) **w/o cascading** — 10-point Gauss–Legendre for every edge (no L1/L2 filtering).
4) **w/o CARM** — oracle metric used directly.
5) **w/o smoothing** — skip post-processing.

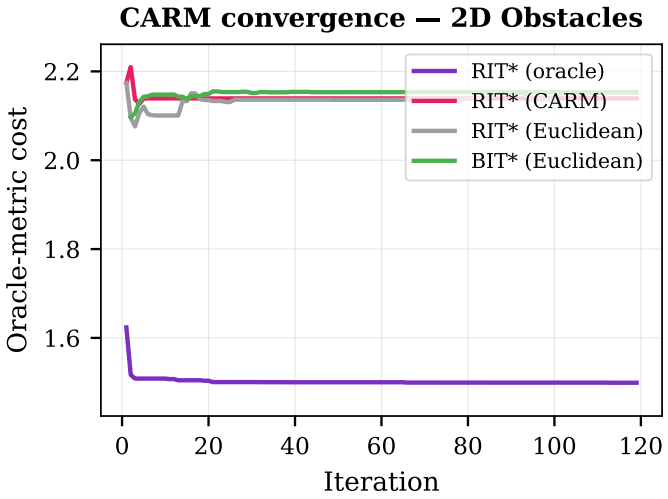


Fig. 8: Convergence comparison under the oracle metric on 2-D Obstacles. RIT* (oracle) converges fastest; CARM achieves lower cost than Euclidean baselines, indicating that the learned metric steers sampling toward better paths.

### E. CARM Analysis

To evaluate CARM independently, all planners are evaluated under the *same* oracle metric (obstacle-inflated) to ensure fair comparison. table IV reports CARM's recovery of the oracle advantage.

CARM recovery varies across environments: it closes 26.5% of the oracle gap on Forest (where obstacle structure is learnable from scattered collision points) but provides minimal benefit on Obstacles and Maze (where the obstacle structure is simpler or the collision distribution is less informative). These results indicate that CARM's effectiveness depends on the informativeness of the collision distribution relative to the true cost landscape.

## V. Discussion and Conclusion

**When does RIT* help?** The benefit scales with $\kappa^{1/d}$ and is maximised under high anisotropy and moderate dimensionality. For isotropic costs ($\kappa \approx 1$), $I_R \approx I_E$ and there is no gain, as confirmed by the Tabletop result. The largest improvements—up to 11% in cost—occur in 6-DOF manipulator planning where $\kappa = 12$.

**Limitations.** (i) The metric cache scales as $\mathrm{res}^d$, which may be prohibitive for $d > 10$; sparse or adaptive caching could address this. (ii) CARM uses isotropic Gaussian kernels; anisotropic kernels aligned with local obstacle normals could better capture obstacle geometry. (iii) CARM recovery is modest on some environments (1.6% on Maze), indicating that additional exploration strategies may be needed to generate informative collision distributions. (iv) The current experiments use simulated environments; real-robot validation is an important direction for future work.

**Conclusion.** We presented RIT*, a Riemannian planning framework that replaces every Euclidean primitive in the batch-informed pipeline with its Riemannian counterpart and

TABLE I: Path cost (mean $\pm$ std, lower is better). **Bold**: best per environment. $^{**}$: RIT* significantly better at $p < 0.01$. $^{*}$: significantly better at $p < 0.05$.

| Environment | RIT* | Inf. RRT* | BIT* | AIT* | EIT* | APT* |
|---|---|---|---|---|---|---|
| 2-D Maze | $4.648 \pm 0.028$ | $4.692 \pm 0.049$ | $\mathbf{4.627 \pm 0.013}$ | $5.522 \pm 0.261$ | $5.553 \pm 0.278$ | $4.724 \pm 0.050$ |
| 2-D Narrow | $\mathbf{1.606 \pm 0.001}^{**}$ | $1.624 \pm 0.005$ | $1.609 \pm 0.002$ | $1.677 \pm 0.036$ | $1.716 \pm 0.037$ | $1.670 \pm 0.047$ |
| 2-D Bug Trap | $\mathbf{1.762 \pm 0.003}$ | $1.768 \pm 0.004$ | $1.751 \pm 0.001$ | $1.878 \pm 0.052$ | $1.909 \pm 0.037$ | $1.769 \pm 0.010$ |
| 3-D Spheres | $3.244 \pm 0.105$ | $3.137 \pm 0.033$ | $\mathbf{2.970 \pm 0.006}$ | $3.259 \pm 0.032$ | $3.193 \pm 0.031$ | $3.319 \pm 0.042$ |
| 3-D Dense Lab | $4.758 \pm 0.337$ | $3.958 \pm 0.022$ | $\mathbf{3.838 \pm 0.003}$ | $4.134 \pm 0.063$ | $4.180 \pm 0.040$ | $4.157 \pm 0.034$ |
| 6-D Tabletop | $\mathbf{1.279 \pm 0.000}$ | $1.279 \pm 0.000$ | $1.279 \pm 0.000$ | $1.279 \pm 0.000$ | $1.279 \pm 0.000$ | $1.279 \pm 0.000$ |
| 6-D Shelf | $\mathbf{1.093 \pm 0.035}^{*}$ | $1.232 \pm 0.021$ | $1.164 \pm 0.061$ | $1.137 \pm 0.031$ | $1.136 \pm 0.033$ | $1.599 \pm 0.536$ |
| 6-D Cluttered | $\mathbf{2.039 \pm 0.082}^{*}$ | $2.278 \pm 0.095$ | $2.162 \pm 0.166$ | $2.131 \pm 0.155$ | $2.163 \pm 0.144$ | $2.570 \pm 0.088$ |

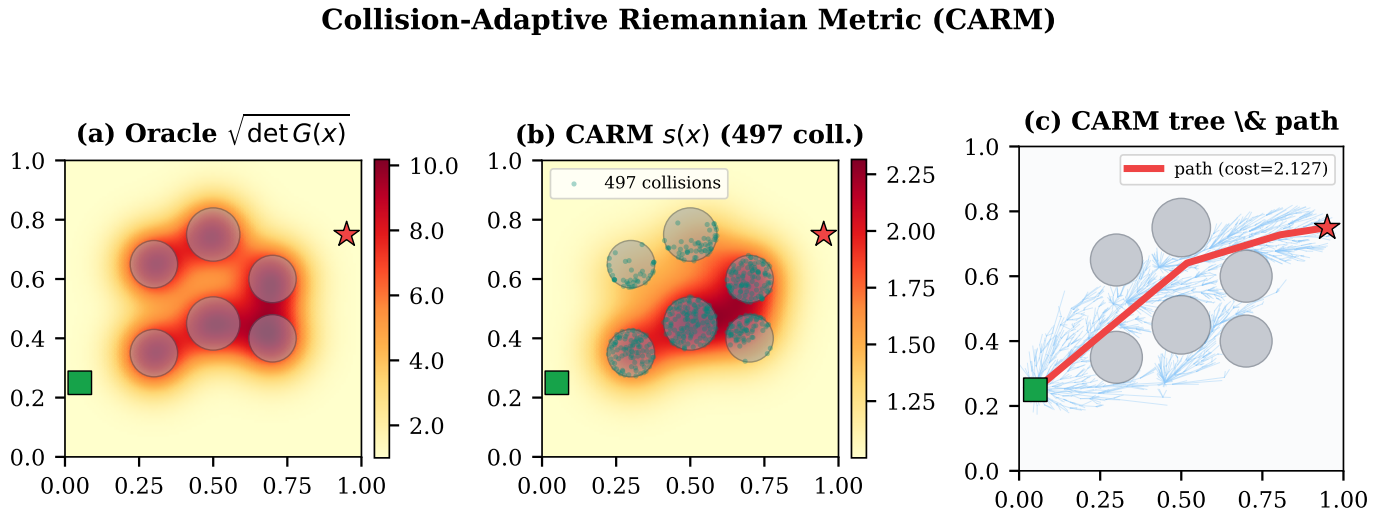


Fig. 9: CARM on 2-D Obstacles. **(a)** Oracle metric field. **(b)** CARM-learned scale $s(x)$ after 100 iterations: the field concentrates near obstacle boundaries. **(c)** Resulting CARM tree and path.

TABLE IV: CARM recovery analysis. All costs evaluated under the oracle metric. Recovery: fraction of the gap from Euclidean to oracle that CARM closes.

| Env. | Oracle | CARM | Euclidean | Recovery |
|---|---|---|---|---|
| 2D Obst. | 1.500 | 2.153 | 2.153 | $-0.1\%$ |
| 2D Forest | 1.732 | 2.726 | 3.084 | 26.5% |
| 2D Maze | 4.572 | 5.103 | 5.112 | 1.6% |

TABLE II: Planning time in seconds (mean $\pm$ std). **Bold**: fastest. All planners achieve 100% success except where noted.

| Env. | RIT* | IRRT* | BIT* | AIT* | EIT* | APT* |
|---|---|---|---|---|---|---|
| 2D Maze | $\mathbf{4.1 \pm 2.2}$ | $55.3 \pm 3.5$ | $293.6 \pm 22.0$ | $7.5 \pm 0.2$ | $5.5 \pm 0.2$ | $61.1 \pm 4.0$ |
| 2D Narrow | $\mathbf{0.5 \pm 0.1}$ | $2.6 \pm 0.0$ | $16.3 \pm 0.2$ | $4.2 \pm 0.1$ | $6.2 \pm 1.6$ | $4.6 \pm 1.0$ |
| 2D Bug | $\mathbf{0.9 \pm 0.2}$ | $16.9 \pm 0.3$ | $229.4 \pm 23.7$ | $5.7 \pm 0.2$ | $2.9 \pm 0.4$ | $15.8 \pm 0.9$ |
| 3D Sph. | $\mathbf{2.0 \pm 0.6}$ | $23.2 \pm 0.8$ | $80.7 \pm 1.2$ | $2.9 \pm 0.1$ | $3.1 \pm 0.1$ | $20.6 \pm 0.4$ |
| 3D Lab | $\mathbf{1.6 \pm 0.9}$ | $26.0 \pm 1.4$ | $88.3 \pm 1.5$ | $3.4 \pm 0.1$ | $3.4 \pm 0.1$ | $28.6 \pm 1.8$ |
| 6D Tab. | $\mathbf{0.2 \pm 0.0}$ | $13.6 \pm 0.2$ | $1.2 \pm 0.0$ | $2.1 \pm 0.0$ | $1.1 \pm 0.0$ | $13.4 \pm 0.3$ |
| 6D Shelf | $1.6 \pm 0.3$ | $14.6 \pm 3.8$ | $\mathbf{1.3 \pm 0.1}$ | $83.3 \pm 36.5$ | $70.1 \pm 31.8$ | $14.7 \pm 4.3$ |
| 6D Clut. | $2.0 \pm 0.5$ | $2.0 \pm 0.3$ | $\mathbf{1.3 \pm 0.0}$ | $1.5 \pm 0.1$ | $1.5 \pm 0.0$ | $6.6 \pm 0.4$ |

TABLE III: Ablation study: path cost for each variant (multi-trial results in progress).

| Variant | 2D Maze | 3D Sph. | 6D Shelf | 6D Clut. |
|---|---|---|---|---|
| RIT* (full) | — | — | — | — |
| w/o Riem. samp. | — | — | — | — |
| w/o cascading | — | — | — | — |
| w/o CARM | — | — | — | — |
| w/o smoothing | — | — | — | — |

learns the cost landscape online via CARM. Multi-trial experiments on 6-DOF UR10e manipulator environments demonstrate statistically significant cost reductions of up to 11% over five baselines. CARM recovers up to 27% of the oracle-metric advantage from collision feedback alone, without any prior cost model.

## References


[1] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.

[2] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.

[3] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," *Int. J. Robot. Res.*, vol. 37, no. 13–14, pp. 1519–1559, 2018.

[4] J. D. Gammell and T. D. Barfoot, "Batch Informed Trees (BIT*): Informed asymptotically optimal anytime search," *Int. J. Robot. Res.*, vol. 39, no. 5, pp. 543–567, 2020.

[5] M. P. Strub and J. D. Gammell, "Adaptively Informed Trees (AIT*) and Effort Informed Trees (EIT*): Asymmetric bidirectional sampling-based path planning," *Int. J. Robot. Res.*, vol. 41, no. 4, pp. 390–417, 2022.

[6] L. Zhang, S. Wang, K. Cai, Z. Bing, F. Wu, C. Wang, S. Haddadin, and A. Knoll, "APT*: Asymptotically optimal motion planning via adaptively prolated elliptical r-nearest neighbors," *IEEE Robot. Autom. Lett.*, 2025.

[7] L. Zhang *et al.*, "Flexible Informed Trees (FIT*): Adaptive batch-size approach in informed sampling-based path planning," in *Proc. IEEE/RSJ IROS*, 2024, pp. 3146–3152.

[8] P. H. Kyaw and J. Kelly, "Geometry-aware sampling-based motion planning on Riemannian manifolds," *arXiv:2602.00992*, 2026.
[9] Y. Zhang, J. Wu, W. Zhang, and Y. Wang, "Bi-AM-RRT*: A bidirectional attractor-guided sampling-based motion planning via assisting metric," *IEEE Trans. Intell. Veh.*, 2023.
[10] H. Klein, N. Jaquier, A. Meixner, and T. Asfour, "On the design of region-avoiding metrics for collision-safe motion generation on Riemannian manifolds," in *Proc. IEEE/RSJ IROS*, 2023.
[11] C.-A. Cheng, M. Mukadam, J. Issac, S. Birchfield, D. Fox, B. Boots, and N. Ratliff, "RMPflow: A computational graph for automatic motion policy generation," in *Proc. WAFR*, 2018.
[12] H. Beik-Mohammadi, S. Hauberg, G. Arvanitidis, G. Neumann, and L. Rozo, "Reactive motion generation on learned Riemannian manifolds," *arXiv:2203.07761*, 2023.
[13] B. Kim, T. T. Um, C. Suh, and F. C. Park, "Tangent bundle RRT: A randomized algorithm for constrained motion planning," *Robotica*, vol. 34, no. 1, pp. 202–225, 2016.
[14] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *Proc. IEEE ICRA*, 2009, pp. 489–494.
[15] M. Zucker *et al.*, "CHOMP: Covariant Hamiltonian optimization for motion planning," *Int. J. Robot. Res.*, vol. 32, no. 9–10, pp. 1164–1193, 2013.
[16] L. Jaillet and T. Siméon, "Path deformation roadmaps," *Int. J. Robot. Res.*, vol. 27, no. 11–12, pp. 1175–1188, 2008.
[17] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016–2021. http://pybullet.org